

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

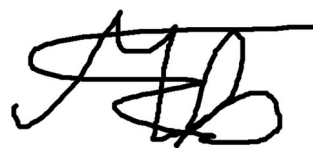
Факультет безопасности информационных технологий

Дисциплина:
«Операционные системы»

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6
«Тестирование функций malloc/free»

Выполнил:

Пахомов Владимир Юрьевич, студент группы N3250



(подпись)

Проверил:

Ханов Артур Рафаэлевич, преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург
2025г.

Содержание

Базовые файловые системы.....	3
Экзотическая файловая система.....	6

Простой и усложнённый варианты

Для тестирования функций malloc/free был написан код на языке C, который выделяет различное количество блоков памяти (для избежания кэширования) и замеряет потраченное время на выделение.

```
benchmark.c:

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <unistd.h>

#define BATCH_SIZE 50

long get_nanoseconds(struct timespec start, struct timespec end) {
    return (end.tv_sec - start.tv_sec) * 1000000000L + (end.tv_nsec -
start.tv_nsec);
}

int main() {
    size_t sizes[] = {
        1024, 2048, 4096, 8192, 16384, 32768, 65536, 100000,
        120000, 131072, 150000, 200000, 262144, 524288, 1048576,
2097152
    };
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);

    int iterations = 100000;
    void *ptrs[BATCH_SIZE];

    printf("Size_Bytes,Avg_Time_NS\n");

    for (int i = 0; i < num_sizes; i++) {
        size_t size = sizes[i];
        long total_time = 0;
        int successful_allocs = 0;

        for (int j = 0; j < iterations / BATCH_SIZE; j++) {
            for (int k = 0; k < BATCH_SIZE; k++) {
                struct timespec start, end;
                clock_gettime(CLOCK_MONOTONIC, &start);
                ptrs[k] = malloc(size);
                clock_gettime(CLOCK_MONOTONIC, &end);

                if (ptrs[k]) {
                    total_time += get_nanoseconds(start, end);
                    successful_allocs++;
                }
            }

            for (int k = 0; k < BATCH_SIZE; k++) {
                if (ptrs[k]) {
                    free(ptrs[k]);
                    ptrs[k] = NULL;
                }
            }
        }
    }
}
```

```

        if (successful_allocs > 0) {
            printf("%zu,%ld\n", size, total_time / successful_allocs);
        }
    }

    return 0;
}

```

Для построения графика зависимости времени от выделения запрашиваемой памяти был написан код на языке Python.

```

graphic.py:
import pandas as pd
import matplotlib.pyplot as plt

df_standard = pd.read_csv('data.csv')
df_jemalloc = pd.read_csv('data_jemalloc.csv')

plt.figure(figsize=(12, 7))

plt.plot(df_standard['Size_Bytes'] / 1024, df_standard['Avg_Time_NS'],
         marker='o', linestyle='-', color='blue', label='Standard malloc
(glibc)', linewidth=2)

plt.plot(df_jemalloc['Size_Bytes'] / 1024, df_jemalloc['Avg_Time_NS'],
         marker='s', linestyle='--', color='red', label='jemalloc
(Facebook)', linewidth=2)

plt.title('Сравнение производительности аллокаторов: glibc vs jemalloc',
fontsize=14)
plt.xlabel('Размер выделяемой памяти (КБ)', fontsize=12)
plt.ylabel('Среднее время выделения (нс)', fontsize=12)

plt.axvline(x=128, color='green', linestyle=':', label='mmap threshold
(128 KB)')

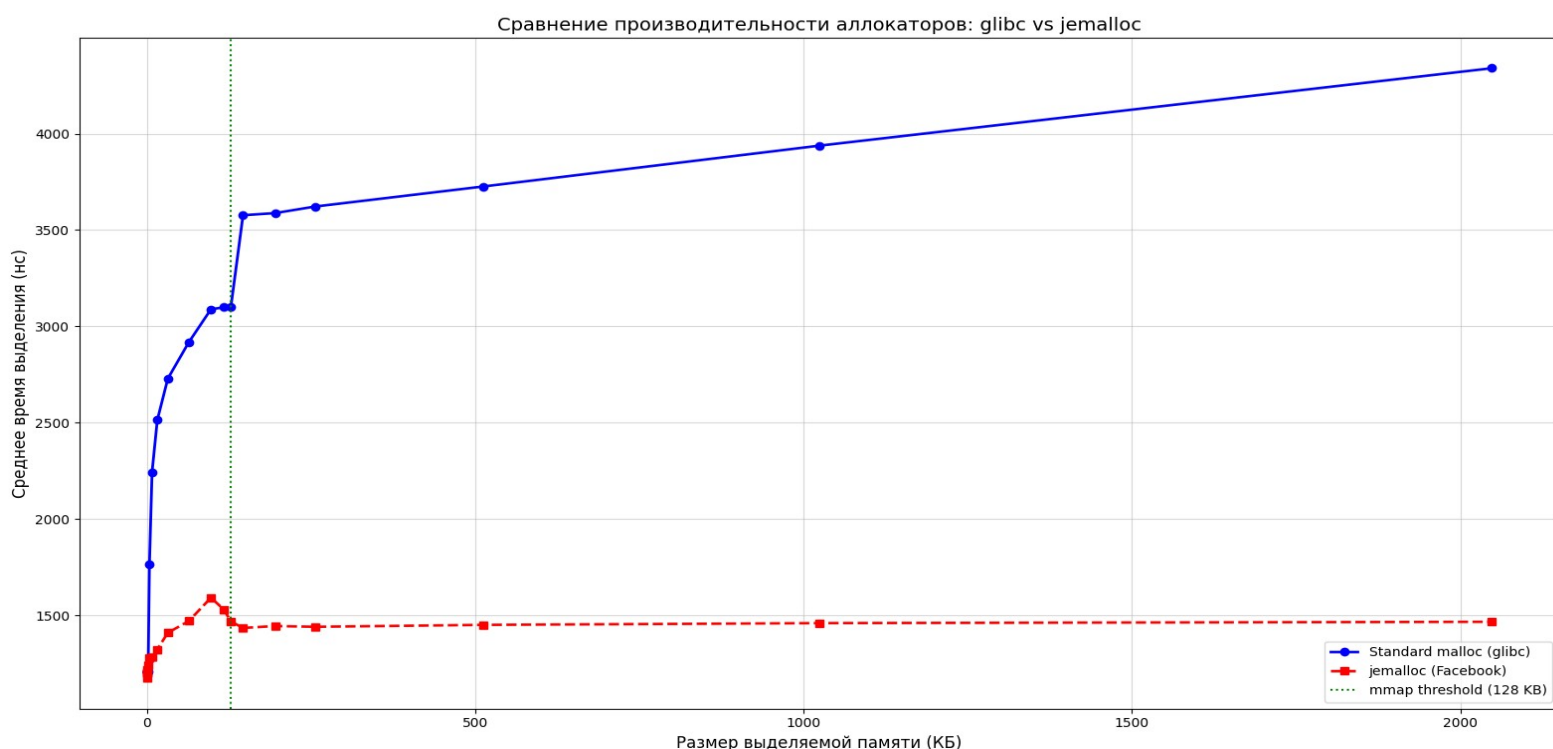
plt.grid(True, which="both", ls="--", alpha=0.5)

plt.legend()

plt.savefig('malloc_comparison.png')
print("График сохранен как malloc_comparison.png")
plt.show()

```

По итогу получаем такой график:



Также я решил провести такой же тест для функции **jemalloc** от компании Facebook*, чтобы сравнить результаты. Установка jemalloc и запуск кода:

```
sudo pacman -S jemalloc
LD_PRELOAD=/usr/lib/libjemalloc.so ./benchmark >
data_jemalloc.csv
```

На графике malloc виден скачок примерно на 128 КБ, который является точкой перехода от системы выделения памяти с brk на mmap. На графике jemalloc эта точка находится на отметке 64 КБ. Также из графика видно, что jemalloc работает гораздо быстрее обычного malloc при больших объемах данных.

Ещё мы получаем такой вывод при вызове команд

```
sudo perf record -ag -e syscalls:sys_enter_mmap -- sleep 30
sudo perf script --header
```

```

# captured on      : Sat Dec 27 20:17:29 2025
# header version   : 1
# data offset      : 600
# data size        : 1750424
# feat offset      : 1751024
# hostname         : vladimir
# os release       : 6.10.2-arch2-1-0.001.gnll
# perf version     : 6.10-1
# arch             : x86_64
# nr_cpus online   : 12
# nr_cpus avail    : 12
# cpudesc          : AMD Ryzen 5 5500U with Radeon Graphics
# cpuid            : AuthenticAMD,23,104,1
# total memory     : 15732964 kb
# cmdline          : /usr/bin/perf
# event            : name = syscalls:sys_enter_mmap, , id = { 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344 }, type = 2 (PERF_TYPE_TRACEPOINT), size = 136, config = 0x64 (syscalls:sys_enter_mmap), id = { 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356 }, type = 1 (PERF_TYPE_SOFTWARE), size = 136, config = 0x9 (PERF_COUNT_SW_DUMMY), { sample_period: 1000000, sample_type: 0 }
# CPU_TOPOLOGY info available, use -I to display
# NUMA_TOPOLOGY info available, use -I to display
# pmu mappings: cpu = 4, amd_tommu_0 = 12, breakpoint = 5, drn_amdgpu = 4294836224, hwmon_acpitz = 4294901761, hwmon_amdgpu = 4294901764, hwmon_bat0 = 4294901762, hwmon_k10temp = 4294901765,
# CACHE info available, use -I to display
# time of first sample : 4540.456257
# time of last sample  : 4568.790317
# sample duration      : 28334.060 ms
# MEM_TOPOLOGY info available, use -I to display
# bpf_prog_info 18: bpf_prog_31efb7c56239148b_restrict_filesystems addr 0xffffffffc0166670 size 297
# bpf_prog_info 19: bpf_prog_feaab199760324c9_sd_devices addr 0xffffffffc0166a2c size 322
# bpf_prog_info 20: bpf_prog_772db7720b2728e9_sd_fw_egress addr 0xffffffffc0166ba4 size 67
# bpf_prog_info 21: bpf_prog_772db7720b2728e9_sd_fw_ingress addr 0xffffffffc0166c20 size 67
# bpf_prog_info 22: bpf_prog_a97c143260cd9940_sd_devices addr 0xffffffffc0169524 size 271
# bpf_prog_info 23: bpf_prog_772db7720b2728e9_sd_fw_egress addr 0xffffffffc0169678 size 67
# bpf_prog_info 24: bpf_prog_772db7720b2728e9_sd_fw_ingress addr 0xffffffffc01696e8 size 67
# bpf_prog_info 25: bpf_prog_772db7720b2728e9_sd_fw_egress addr 0xffffffffc016c448 size 67
# bpf_prog_info 26: bpf_prog_772db7720b2728e9_sd_fw_ingress addr 0xffffffffc016c4d8 size 67
# bpf_prog_info 27: bpf_prog_a97c143260cd9940_sd_devices addr 0xffffffffc016e94c size 271
# bpf_prog_info 28: bpf_prog_a97c143260cd9940_sd_devices addr 0xffffffffc01718a0 size 271
# bpf_prog_info 29: bpf_prog_71bba91db5466642_sd_devices addr 0xffffffffc0171b98 size 493
# bpf_prog_info 30: bpf_prog_772db7720b2728e9_sd_fw_egress addr 0xffffffffc0171de0 size 67
# bpf_prog_info 31: bpf_prog_772db7720b2728e9_sd_fw_ingress addr 0xffffffffc0171e4c size 67
# bpf_prog_info 35: bpf_prog_98b7b6f166b26c8a_sd_devices addr 0xffffffffc017d738 size 114
# bpf_prog_info 39: bpf_prog_772db7720b2728e9_sd_fw_egress addr 0xffffffffc0183d5c size 67
# bpf_prog_info 40: bpf_prog_772db7720b2728e9_sd_fw_ingress addr 0xffffffffc0183ddc size 67
# bpf_prog_info 41: bpf_prog_c1591dae5f5bcd9f_sd_devices addr 0xffffffffc01873c8 size 301
# bpf_prog_info 42: bpf_prog_772db7720b2728e9_sd_fw_egress addr 0xffffffffc0187578 size 67
# bpf_prog_info 43: bpf_prog_772db7720b2728e9_sd_fw_ingress addr 0xffffffffc01875f8 size 67
# bpf_prog_info 48: bpf_prog_772db7720b2728e9_sd_fw_egress addr 0xffffffffc0178988 size 67
# bpf_prog_info 49: bpf_prog_772db7720b2728e9_sd_fw_ingress addr 0xffffffffc0178a08 size 67
# btf info of id 38
# cpu pmu capabilities: max_precise=0
# AMD systems uses ibs op// PMU for some precise events, e.g.: cycles:p, see the 'perf list' man page for further details.
# missing features: BRANCH_STACK GROUP_DESC AUXTRACE STAT CLOCKID DIR_FORMAT COMPRESSED CLOCK_DATA HYBRID_TOPOLOGY
# =====
sleep 9035 [001] 4540.456257: syscalls:sys_enter_mmap: addr: 0x00000000, len: 0x0002ecd3, prot: 0x00000001, flags: 0x00000002, fd: 0x00000003, off: 0x00000000
7faf3e815dfc [unknown] (/usr/lib/ld-linux-x86-64.so.2)
7faf3e7fd939 [unknown] (/usr/lib/ld-linux-x86-64.so.2)
7faf3e806fec [unknown] (/usr/lib/ld-linux-x86-64.so.2)
7faf3e7fb1b6 [unknown] (/usr/lib/ld-linux-x86-64.so.2)
7faf3e7f483d [unknown] (/usr/lib/ld-linux-x86-64.so.2)
7faf3e7f3456 _dl_catch_exception+0xa6 (/usr/lib/ld-linux-x86-64.so.2)

```

В строке

```
sleep 9035 [001] 4540.456257: syscalls:sys_enter_mmap: addr:
0x00000000, len: 0x0002ecd3, prot: 0x00000001, flags:
0x00000002, fd: 0x00000003, off: 0x00000000
```

Видно, что примерно при 187 КБ система уже использует mmap.